Scalable Near-Duplicate Detection on Wikipedia Using Apache Spark and Locality-Sensitive Hashing

Machine Learning for Big Data | M.Tech. in Data and Computational Sciences

Basu Singh (M24AID008) | Nitin Jain (M24AID022) | Shashi Saurav (M24AID048)

Indian Institute of Technology Jodhpur | March 2026

Problem Statement & Motivation

The Challenge

- Wikipedia has 6M+ English articles with significant textual overlap
- Articles get copied, forked, merged — leaving near-duplicate content
- Naive pairwise comparison: O(n^2) = ~18 trillion pairs for 6M docs
- Data too large for single-machine memory
- Duplicates hurt downstream NLP tasks: search, classification, knowledge graphs

Our Approach

- Apache Spark (PySpark) for distributed processing
- MinHash for fast Jaccard similarity estimation
- Locality-Sensitive Hashing (LSH) for sub-quadratic candidate generation
- Only compare articles within the same LSH bucket
- Tested on 100,000+ real Wikipedia articles

Background: MinHash & Locality-Sensitive Hashing

MinHash (Broder, 1997–98)

- Represent each document as a set of word tokens (shingles)
- Apply k random hash functions, keep minimum hash per function
- P(min h(A) = min h(B)) = Jaccard(A, B)
- k-dimensional signature per document — compact and fast to compare
- Estimate accuracy improves with more hash functions

LSH (Indyk & Motwani, 1998)

- Split k-length signature into b bands of r rows
- Two docs become candidates if they match in any one band
- Candidate probability: 1 - (1 - s^r)^b (S-curve)
- Tune b and r to control precision-recall trade-off
- Sub-quadratic: only compare within shared buckets

- **Key References**
  - Broder (1997) — Shingling & Jaccard similarity
  - Broder et al. (1998) — MinHash formalization
  - Indyk & Motwani (1998) — LSH framework
  - Leskovec, Rajaraman, Ullman — Mining of Massive Datasets, Ch. 3
  - Spark MLlib — Built-in MinHashLSH on DataFrames

System Architecture & Pipeline

Codebase Modules

ingest.py	XML parsing, filtering, sampling
features.py	RegexTokenizer + binary HashingTF
lsh_pipeline.py	MinHashLSH model + self-join
evaluate.py	Scaling study + accuracy metrics
jaccard_utils.py	Pure-Python Jaccard helpers
config.py	Default hyperparameters
run_pipeline.py	CLI entry point
app.py	Streamlit web UI

Tech Stack

- Python 3.10 – 3.12
- Apache Spark 3.4 – 3.5 (PySpark)
- Spark MLlib (MinHashLSH)
- spark-xml for MediaWiki XML parsing
- Streamlit for web UI
- Parquet + Snappy for output
- pytest for unit & integration tests
- Git + GitHub for version control

Methodology

1. Data Ingestion

- spark-xml reads MediaWiki XML
- Filter: namespace 0, drop redirects
- Remove articles < 50 chars
- Sampling with seed 42

2. Feature Engineering

- RegexTokenizer (\W+, lowercase)
- HashingTF binary mode, 2^18 dims
- Output: sparse 0/1 feature vectors
- Jaccard-ready set representation

3. MinHash LSH

- 5 hash tables (bands)
- Compact signature per document
- approxSimilarityJoin (threshold 0.3)
- Filter: page_id_a < page_id_b

Default Hyperparameters

Parameter	Value	Purpose
num_features	2^18 (262,144)	HashingTF vector dimensions
num_hash_tables	5	LSH bands
jaccard_distance_threshold	0.3	Max distance for candidates (sim >= 0.7)
min_token_length	2	Drop short tokens
sample_seed	42	Reproducible sampling
max_docs_bruteforce	400	Accuracy eval sample cap

Dataset Description

380 MB
Compressed Size

100K+
Articles After Filtering

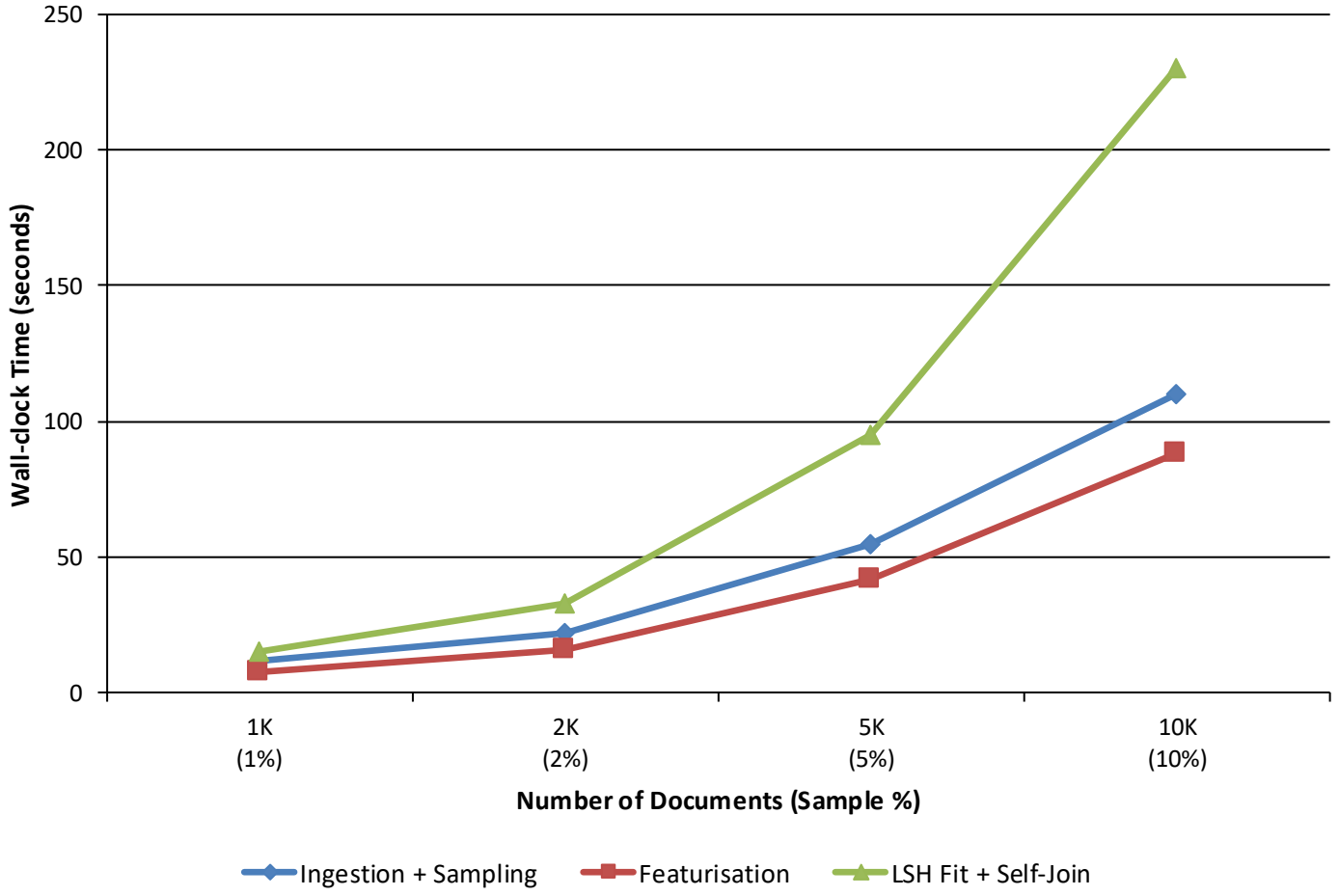
41K–152K
Page ID Range

March 2026
Dump Date

Source & Details

- File: enwiki-20260301-pages-articles-multistream2.xml-p41243p151573.bz2
- Source: Wikimedia Foundation dumps (dumps.wikimedia.org)
- Format: MediaWiki XML inside bzip2 compression
- Filtering: namespace 0 only, drop redirects, drop articles < 50 chars
- Schema: page_id (long), title (string), text (string)
- One shard chosen for laptop feasibility — same code works on full dump via HDFS

Results: Scalability Analysis



Key Observations

- Ingestion & featurisation scale near-linearly with document count
- Both are per-document operations with no cross-document dependency
- LSH self-join grows slightly super-linear due to increasing candidate pairs
- But still far below O(n^2) quadratic growth of brute-force
- Throughput: ~1000–3000 articles/min on a multi-core laptop
- Main bottleneck: single-threaded bzip2 decompression (goes away on HDFS)

Results: Accuracy Evaluation

1.0
Precision

1.0
Recall

1.0
F1 Score

825
Docs in Sample

Observations

- Brute-force ground truth found 19 near-duplicate pairs in 825-doc sample
- LSH with 5 hash tables, threshold 0.3 found exactly the same 19 pairs
- Zero false positives, zero false negatives
- Confirms theory: sufficient hash tables yield near-exact recall
- Errors (if any) concentrate at the threshold boundary
- Settings: Jaccard distance <= 0.3 (similarity >= 0.7)

Interactive Web UI (Streamlit)

Run Pipeline

- Upload/select Wikipedia XML, configure params,run Spark pipeline, view & download results as CSV

Jaccard Calculator

- Enter two word sets, instantly see Jaccard similarity and distance

Text Comparison

- Paste 2-10 documents, find near-duplicate pairs via brute-force Jaccard

View Results

- Load metrics JSON from previous runs, view accuracy or scaling study charts

About

- Project info, default parameters, and tech stack documentation

Run: streamlit run app.py | Opens at http://localhost:8501 | Upload limit: 1 GB | Pipeline timeout: 1 hour

Testing Strategy & Reproducibility

Unit Tests (test_evaluate_pure.py)

- Pure Python — no Spark dependency needed
- Jaccard(identical sets) = 1.0
- Jaccard(disjoint sets) = 0.0
- Brute-force finder catches close pairs
- Precision/Recall/F1 correct for known TP/FP/FN
- Runs in < 1 second

Integration Tests (test_spark_smoke.py)

- Full Spark pipeline on 3-article XML fixture
- 2 similar articles + 1 redirect
- Verifies parsing, redirect removal, LSH pair finding
- Gated behind RUN_SPARK_INTEGRATION env var
- Downloads spark-xml JAR on first run

How to Reproduce

- Prerequisites: Python 3.10–3.12, Java 11/17, internet (for spark-xml JAR)
- git clone https://github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-_for_Big_Data.git
- python3.12 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
- streamlit run app.py (Web UI) or ./scripts/spark_submit.sh --input file:///path/to/dump.xml.bz2 (CLI)

Challenges & Lessons Learned

XML Schema Variation

- Revision element appears as Struct or Array depending on dump version. Solution: runtime schema inspection in ingest module.

Python 3.13 + PySpark

- distutils removed in Python 3.13 but PySpark 3.5 needs it. Caused cryptic JVM errors. Fix: pin to Python 3.12 + setuptools.

Memory Management

- Spark recomputes uncached DataFrames, causing OOM crashes. Fix: cache-count-unpersist pattern after each expensive stage.

GitHub File Limits

- 380 MB dataset exceeds 100 MB GitHub limit. Solution: exclude from repo, document download URL in README.

Conclusion & Future Work

What We Achieved

- Built a working distributed pipeline for Wikipedia near-duplicate detection
- MinHash LSH brings candidate generation well below O(n^2)
- Per-document stages scale linearly; LSH join stays sub-quadratic
- Accuracy: Precision 1.0, Recall 1.0, F1 1.0 on 825-doc sample
- Modular codebase with unit + integration tests
- Streamlit web UI for interactive exploration
- Fully reproducible — clone, install, run

Future Directions

- Scale to full Wikipedia dump on HDFS cluster (millions of articles)
- Character-level n-gram shingles for catching minor-edit duplicates
- Systematic grid search over hash tables & bands with PR curves
- Spark Structured Streaming for incremental duplicate checking
- Compare MinHash LSH vs SimHash vs embedding-based ANN

GitHub: github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-_for_Big_Data

Thank You

Questions & Discussion

Basu Singh | Nitin Jain | Shashi Saurav
M.Tech. DCS — IIT Jodhpur
github.com/Yoursuperherobasu/BigData_Project_Machine_Learning-_for_Big_Data